



The Daily Brief — Podcast Studio

Project Documentation, Presentation Guide & FAQ

Ironhack – Ioanna Renta | May 2026

1. Project Overview

The Daily Brief is an automated podcast studio that transforms any news article URL or live Hacker News top stories into a fully produced audio episode in under 30 seconds. The user pastes a link or selects a number of stories, clicks one button, and receives a broadcast-ready script and an MP3 file ready to play or download.

Pipeline

Article URL tab: URL → Extract text → GPT-4o-mini script → OpenAI TTS → MP3

Hacker News tab: HN API → Top N stories → GPT-4o-mini script → OpenAI TTS → MP3

2. Project Structure

```
podcast-studio/
├── src/
│   ├── data_processor.py    # Fetches and extracts text from article URLs
│   ├── llm_processor.py     # Transforms content into podcast script via
GPT-4o-mini
│   ├── tts_generator.py     # Converts script to audio via OpenAI TTS
│   ├── hn_processor.py      # Fetches top stories from Hacker News API
│   └── main.py              # Gradio interface – connects the full pipeline
├── requirements.txt
├── README.md
└── .env                    # API keys (not committed)
```

3. Setup & How to Run

Install dependencies

```
pip3 install -r requirements.txt
```

Create your .env file

```
OPENAI_API_KEY=sk-proj-...
```

Run the app

```
cd src && python3 main.py
```

Then open **<http://127.0.0.1:7860>** in your browser.

4. How Each Module Works

data_processor.py — Article Fetcher

Takes a URL, fetches the webpage using the Requests library, and strips out all noise (ads, navigation, footers) using BeautifulSoup4. Returns a clean dictionary with the article title and body text, capped at 5,000 characters to stay within API token limits. Handles timeouts, broken links, and connection errors gracefully.

hn_processor.py — Hacker News API

Connects to the free public Hacker News Firebase API — no API key required. Fetches the top N story IDs, retrieves details for each story (title, URL, score, author), and combines them into a single content dictionary ready to pass to the LLM processor. The key API endpoints are:

- **Top stories list:** <https://hacker-news.firebaseio.com/v0/topstories.json>
- **Story details:** <https://hacker-news.firebaseio.com/v0/item/{id}.json>

Hacker News API — Code Snippet

```
# Fetch top story IDs (returns a list of integers)
response = requests.get(
    "https://hacker-news.firebaseio.com/v0/topstories.json"
)
story_ids = response.json()[:5] # top 5 IDs

# Fetch details for one story
story = requests.get(
    f"https://hacker-news.firebaseio.com/v0/item/{story_ids[0]}.json"
).json()
```

llm_processor.py — Script Generator

Sends the article or HN briefing content to GPT-4o-mini with a carefully engineered prompt that instructs the model to behave like a professional podcast host named The Daily Brief. The prompt enforces conversational language, a consistent structure (welcome → story → sign-off), and a target length of 150-200 words — approximately 90 seconds when read aloud.

tts_generator.py — Audio Generator

Sends the generated script to OpenAI's TTS API using the alloy voice and tts-1 model. The audio response is written directly to an MP3 file using response.content. Typical output is 60-90 seconds of natural-sounding speech.

main.py — Gradio Interface

Wires all modules together in a two-tab Gradio interface. Tab 1 accepts a news article URL. Tab 2 presents a slider to choose how many Hacker News stories to include (3-

10). Both tabs output the generated script as text and the audio as a playable/downloadable player.

5. Tech Stack

- **Backend:** Python 3.8+
- **Frontend:** Gradio
- **Article extraction:** BeautifulSoup4 + Requests
- **News API:** Hacker News Firebase API (free, no key required)
- **LLM:** OpenAI GPT-4o-mini
- **Text-to-Speech:** OpenAI TTS (voice: alloy)

6. Presentation Guide (5-7 minutes)

Introduction — Who I am and what I built (1 min)

I built The Daily Brief — an automated podcast studio that turns any news article or live Hacker News feed into a broadcast-ready audio episode in under 30 seconds. One URL, one click, one podcast.

Problem — Why this matters (1 min)

Most people do not have time to read the news. Audio is more accessible — you can consume it while commuting, exercising, or cooking. But producing a podcast manually takes hours of recording, editing, and publishing. This system does it in seconds, for any article, on demand, at near-zero cost.

Demo — Live walkthrough (3 min)

1. Show the Gradio UI at <http://127.0.0.1:7860>
2. Paste a live BBC News article URL and click Generate Podcast
3. Show the script appearing and play the audio
4. Switch to the Hacker News tab, set slider to 5, generate
5. Play the HN brief and show the tech headlines it picked up

Takeaways — What I learned and what I would add next (1 min)

- Prompt engineering matters — the structure of the LLM prompt determines the quality of the output more than anything else
- Modular code pays off — having four independent modules made debugging fast and the HN extension trivial to add
- What I would add: multiple voice options, multi-story episodes from URLs, episode history, RSS feed support, two-host conversation format

7. FAQ

Q: What happens if the article is behind a paywall?

BeautifulSoup can only read publicly accessible content. Paywalled articles return a clear error message to the user. A future fix would be to also accept pasted text as an alternative input alongside the URL.

Q: Does the Hacker News integration need an API key?

No. The Hacker News Firebase API is completely free and public. No account, no key, no rate limit concerns for this use case. It is one of the cleanest public APIs available.

Q: How long does it take to generate an episode?

About 20-30 seconds end to end — roughly 5 seconds to fetch the article, 10 seconds for the LLM, and 10 seconds for TTS. The Hacker News tab is slightly faster since it skips web scraping.

Q: How much does it cost per episode?

Very little. GPT-4o-mini costs fractions of a cent per call. OpenAI TTS costs \$15 per million characters — a 200-word script is roughly 1,000 characters, so approximately \$0.015 per episode.

Q: Why GPT-4o-mini and not GPT-4o?

For this task — summarising and reformatting a news article — GPT-4o-mini produces excellent results at a fraction of the cost and with faster response times. GPT-4o would be overkill and slower.

Q: Why Gradio instead of a full web app?

Gradio builds a functional, professional-looking UI in about 20 lines of Python with no frontend knowledge required. For a prototype and demo it is the fastest path from idea to working interface. For production, a FastAPI + React stack would be the next step.

Q: Could this work for languages other than English?

Yes — both GPT-4o-mini and OpenAI TTS support multiple languages. The prompt would need to specify the target language and the voice selection may need updating for optimal results.

Q: What is the difference between this and NotebookLM?

NotebookLM generates long-form conversational podcasts from documents you upload. The Daily Brief is focused on news — short, punchy, on-demand briefings from live URLs or a real-time API. Different use case, simpler setup, faster output.

Q: Is this production-ready?

It is a solid MVP. For production you would add user authentication, a database for episode history, async processing for longer articles, rate limiting, and a hosted deployment rather than running locally.